AMRITA VISHWA VIDYAPEETHAM

FINAL PROJECT REPORT

AI-Powered RF Digital Twin Using ADALM-Pluto

Automatic modulation classification on real over-the-air signals

23AID203 – Software-Defined Communication Systems

Faculty: **Kavitha K**

Group A-16

B. Suryacharantej – CB.AI.U4AID25009
B. Sai Srikar – CB.AI.U4AID25006
P. Sathwik – CB.AI.U4AID25041
M. Neeraj – CB.AI.U4AID25031

Amrita Vishwa Vidyapeetham

October 2026

Abstract

Every radio signal carries its information in a particular style, called its modulation, and a receiver has to know that style before it can read the message. In this project we built a small convolutional neural network (CNN) that looks at the raw I/Q samples of a signal and tells us which of five modulations it is: AM-DSB, WBFM, BPSK, QPSK or QAM16.

We first trained the network on the RadioML 2016.10a dataset, where it reached 85.25% on the test set. We then moved it onto real hardware. Using a single ADALM-Pluto with two antennas, we transmitted all five modulations through the air and fed the received samples to the same network. Its accuracy dropped to 74.1%, which showed us clearly how different real signals are from simulated ones. To fix this we recorded 25,000 real frames and fine-tuned the network on them, using separate training, validation and test sets in a 70/15/15 split. On a completely new recording of 9,600 frames that the network had never seen, it was correct 96.7% of the time. We also built a live dashboard in MATLAB that recognises all five modulations in real time.

1. Introduction

Wireless devices around us, such as FM radios, mobile phones, Wi-Fi routers and GPS receivers, all send information by changing a carrier wave in some way. They might change its amplitude (AM), its frequency (FM), its phase (BPSK and QPSK), or the amplitude and phase together (QAM). Whatever the method, the receiver has to know it in advance, much like you need to know the language of a letter before you can read it.

In many situations the receiver does not know this in advance. Spectrum monitoring, cognitive radio and defence systems regularly pick up unknown signals. Automatic Modulation Classification (AMC) is the task of identifying the modulation directly from the received signal, without being told. A received signal is usually described by its in-phase (I) and quadrature (Q) components, two numbers at every instant that act like the x and y position of a point. Deep learning has become popular for AMC because a network can learn the patterns in these I/Q samples by itself, without anyone designing features by hand.

While reading the literature, we noticed that most work tests its models only on simulated datasets. Real hardware adds things that simulations do not fully capture: noise, a random phase rotation, reflections, the radio's own filtering, IQ imbalance and a DC offset. This led to the question at the centre of our project:

Problem statement: How can an unknown RF signal be identified automatically and reliably, not only in simulation but on real radio hardware?

2. Objectives

We set ourselves six objectives. The first five came from our mid-review plan. We added the sixth once we saw how much the accuracy dropped on real signals.

#	Objective	What we achieved
1	Train a CNN on raw 2×128 I/Q samples, with separate training, validation and test sets	85.25% test accuracy on RadioML (70/15/15 split)

2	Configure the ADALM-Pluto to transmit and receive all five modulations over the air	Radio link verified with a test tone; 25,000 real frames recorded
3	Capture real-time I/Q samples and give them to the CNN	Received samples converted to the same 2×128 format as the dataset
4	Classify live signals in a real-time dashboard	All five modulations recognised correctly in the live demo
5	Compare the dataset results with the hardware results	85.25% on the dataset against 74.1% on real signals
6	Close the gap by fine-tuning the CNN on real Pluto data	96.7% on an independent fresh recording

3. Literature Survey

We studied four recent papers on modulation classification. All of them report strong results, but each leaves something open, either because it stays on a dataset or because it relies on a fairly complex model or receiver.

Paper (year)	Platform	Method	Findings and gap
Automatic Modulation Classification with Deep Neural Networks (2023)	MDPI – Electronics	CNNs with dilated convolution, statistics pooling and squeeze-and-excitation	**Findings:** strong results on RadioML 2018.01A, best model 98.9%. **Gap:** focuses on the model and the dataset, not on real-time hardware.
Real-time Modulation Classification Architecture in Software Defined Radio (2024)	ScienceDirect – Physical Communication	Low-complexity 1D CNN with an SDR-based receiver	**Findings:** real-time AMC on an SDR with fast inference. **Gap:** the receiver is quite complex; a simpler AI-to-hardware workflow is still needed.
Modulation Recognition Based on Multi-modal Fusion of ResNet and Traditional Signal Processing (2025)	IEEE Xplore – IEEE MTT-S IWS	ResNet-18 combined with traditional signal processing, tested on ADALM-PLUTO	**Findings:** over 98% with ResNet-18, faster than ResNet-50. **Gap:** a complex multimodal approach compared with a simple CNN pipeline.
A Novel Deep Learning Automatic Modulation Classifier with Fusion of Multichannel Information Using GRU (2023)	Springer Nature – Journal on Wireless Communications and Networking	GRU network fusing I/Q with amplitude and phase	**Findings:** better accuracy at low SNR on RadioML with 128-sample inputs. **Gap:** the extra feature channels make the model harder to deploy.

From this we saw a clear gap. There was room for a simple method that is first checked on a dataset and then shown to work on real hardware. That is what we set out to build: a small 1D CNN that

uses only raw 2×128 I/Q samples, validated on RadioML and then deployed and fine-tuned on a real ADALM-Pluto.

4. Methodology: The RF Digital Twin

We call the system a digital twin because the same network runs in two places. In the virtual twin it works on simulated signals from the RadioML dataset. In the physical twin it works on real signals sent and received by the ADALM-Pluto. Because the network is identical in both, any difference in accuracy tells us how far real signals are from simulated ones.

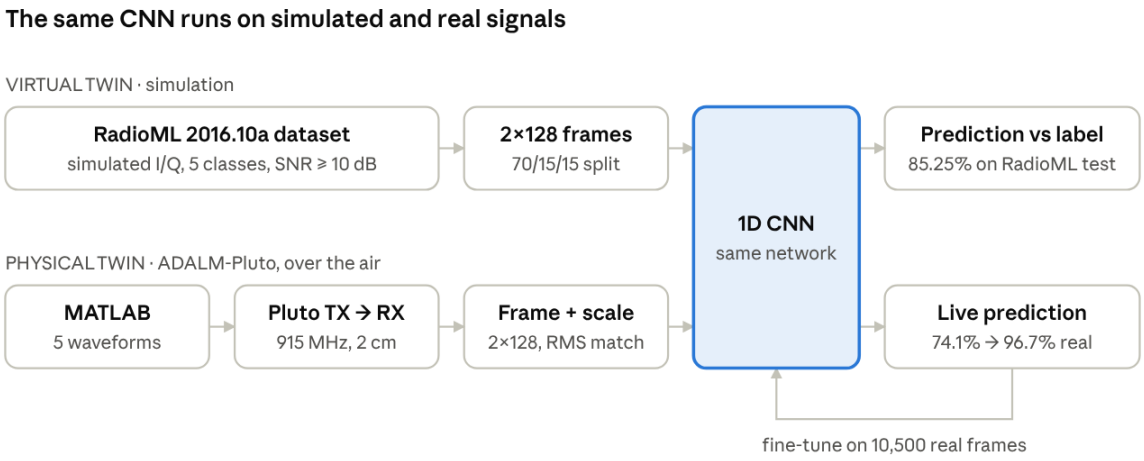


Figure 1. The RF digital twin: both twins feed the same CNN, and real frames are used to fine-tune it.

The work went in this order. We trained the CNN on RadioML. We set up the Pluto and checked the radio link. We tested the RadioML-trained network on real signals and measured the drop. We recorded a real dataset and fine-tuned the network on it. Finally, we tested the fine-tuned network on a fresh recording and ran it live.

5. Hardware Setup

We used a single ADALM-Pluto for both transmitting and receiving. It can do both at the same time, which is called full duplex. One antenna is screwed onto the TX1 port and the other onto RX1, about 2 cm apart, so every signal in this project really travels through the air. Since the transmitter and receiver share one clock, there is almost no frequency mismatch between them. The air path still adds real noise, phase rotation and reflections.

Component	Details
ADALM-Pluto SDR	Analog Devices AD9363 transceiver, 325 MHz to 3.8 GHz, one transmitter and one receiver
Antennas	Two SMA antennas on TX1 and RX1, about 2 cm apart
USB cable	Connected to the middle (data) micro-USB port; carries the I/Q samples and powers the Pluto
Software	MATLAB R2026b with the Communications, DSP System, Signal Processing and Deep Learning toolboxes

Support package	Communications Toolbox Support Package for ADALM-PLUTO Radio, which provides *sdrtx*, *sdrrx* and *findPlutoRadio*

Parameter	Value	Why we chose it
Centre frequency	915 MHz	Well inside the Pluto's range
Sample rate	1 MS/s	One million I/Q samples per second
Samples per symbol	8	Same as the RadioML dataset
Symbol rate	125 kbaud	1 MS/s divided by 8
Pulse shaping	RRC, roll-off 0.35	Keeps the bandwidth limited and matches RadioML
TX gain	−30 dB	The antennas are very close, so a low power avoids overloading the receiver
RX gain	30 dB (manual)	A strong signal of about 90 ADC steps without clipping
Samples per read	8,192	Gives 64 CNN frames per read

Before involving the AI at all, we checked that the radio link worked. We transmitted a pure 100 kHz tone and looked at the received spectrum. The tone arrived at exactly 100.0 kHz, about 80 dB above the noise floor, with no clipping. The small spike at −100 kHz is a weak mirror image of the tone, about 40 dB lower, which is normal for this radio chip.

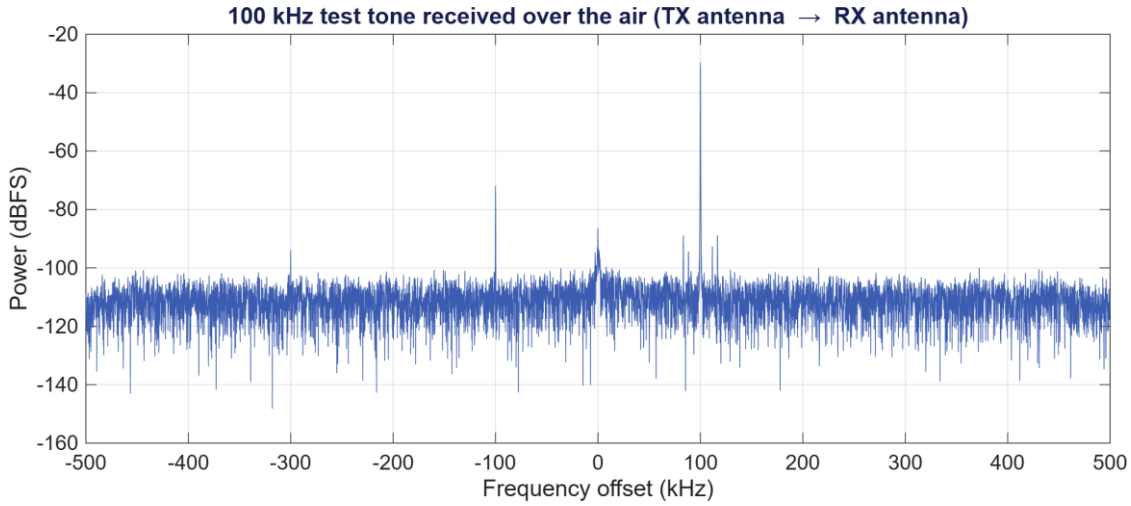


Figure 2. Spectrum of the 100 kHz test tone received over the air.

6. Generating the Five Modulations

We generated all five signals in MATLAB in the same way the RadioML dataset was made, so that the network would see familiar signals. The three digital modulations start from random bits and are smoothed with a root-raised-cosine (RRC) filter at 8 samples per symbol. The two analog modulations carry a short music clip that comes built into MATLAB. Each waveform is scaled to a peak of 0.7 and played on repeat by the Pluto.

Modulation	Type	How we generated it	Bits per symbol	Bit rate

AM-DSB	Analog	The music changes the amplitude: 1 + 0.8·m(t), carrier included	–	–
WBFM	Analog	The music changes the frequency, with 20 kHz peak deviation	–	–
BPSK	Digital	Random bits mapped to 2 phases, then RRC filtering	1	125 kbps
QPSK	Digital	Random bits mapped to 4 phases, then RRC filtering	2	250 kbps
QAM16	Digital	Random bits mapped to a 4×4 grid, then RRC filtering	4	500 kbps

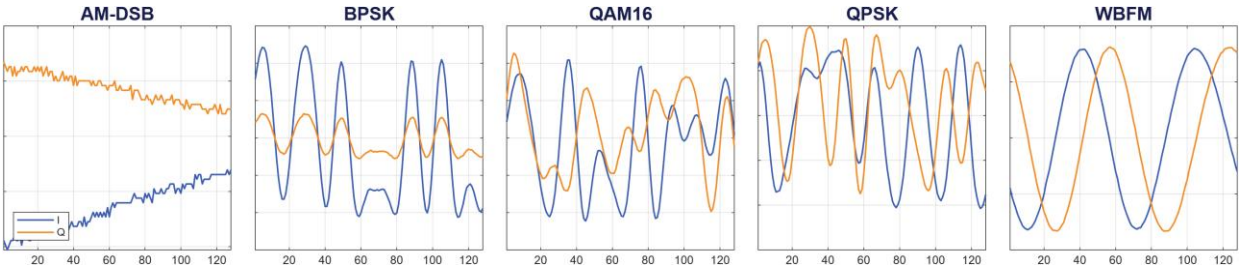


Figure 3. One real received frame of each modulation (I in blue, Q in orange), exactly as the CNN sees it.

7. Receiver Processing

The receiver side has one job: turn the stream of received samples into inputs that look like the dataset. For every read from the Pluto we do the following.

1. **Read a buffer** of 8,192 complex samples, which is 8.2 ms of signal.
2. **Remove the DC component** by subtracting the average, which removes the leakage from the radio's own oscillator.
3. **Cut the buffer** into 64 frames of 128 samples, the same frame length as RadioML.
4. **Scale each frame** to an RMS of 0.00812, the average level of the RadioML training frames. This lets one network handle both strong and weak signals.
5. **Stack I and Q** into a 2×128 matrix, with I in the first row and Q in the second.
6. **Classify** each frame with the CNN. The live dashboard averages the scores over the 64 frames of a buffer.

One side effect matters later. Removing the DC component also removes the carrier of the AM-DSB signal, so our real AM-DSB frames look noticeably different from the ones in RadioML.

When we plot the real received samples, each modulation forms its own shape. BPSK makes a line, QPSK four clusters, QAM16 a cloud of sixteen levels, WBFM a ring because its amplitude stays constant, and AM-DSB a short line along the amplitude direction. All of them are slightly rotated by the channel. These shapes are what the CNN learns to tell apart.

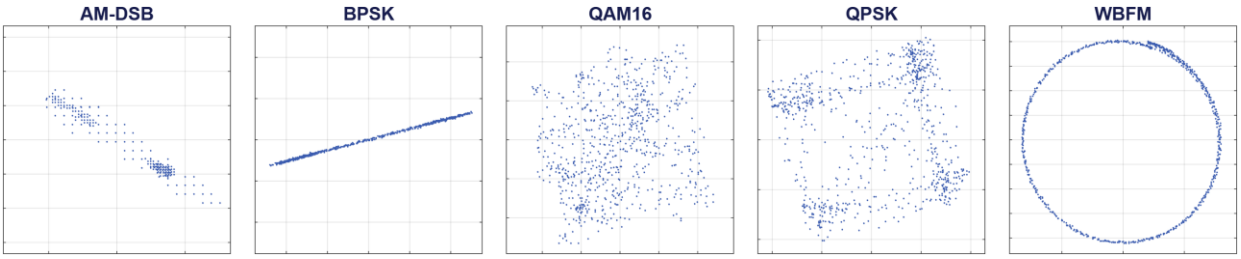


Figure 4. Real constellations captured by the ADALM-Pluto at the strongest power setting.

8. CNN Architecture

We kept the network deliberately small. It has about 11,000 learnable parameters, which is enough for the task and runs comfortably in real time on a laptop without a graphics card.

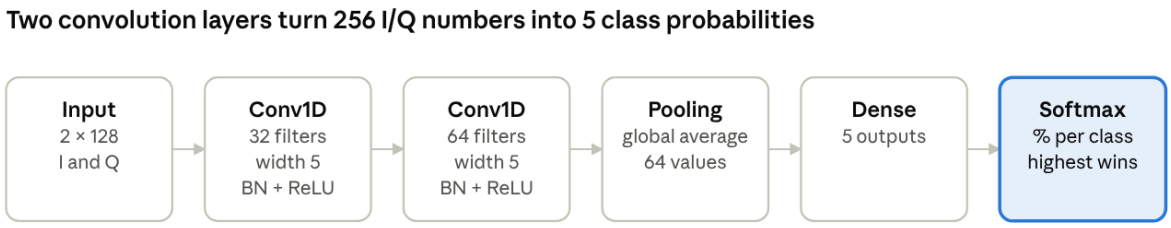


Figure 5. The 1D CNN, from the 2×128 input to the five class probabilities.

Each convolution filter is a small pattern detector, five samples wide, that slides along the signal in time. The first layer has 32 such filters and picks up simple patterns such as jumps and smooth curves. The second layer has 64 filters and combines those into more complex patterns. Batch normalisation keeps the values in a stable range during training, and ReLU keeps only the useful positive responses. Global average pooling then summarises the whole frame into 64 numbers, the fully connected layer turns them into five scores, and softmax converts the scores into probabilities. The class with the highest probability is the answer.

Layer	Output size	Learnable parameters
Sequence input	2 × 128	0
Conv1D (32 filters, width 5) + batch norm + ReLU	32 × 128	352 + 64
Conv1D (64 filters, width 5) + batch norm + ReLU	64 × 128	10,304 + 128
Global average pooling	64	0
Fully connected	5	325
Softmax and classification	5	0
Total		**11,173**

9. Training, Validation and Testing

In both training stages we split the data into three separate parts, so that the reported accuracy is honest.

- **Training (70%)** is the data the network learns from.
- **Validation (15%)** is checked every 50 training steps. When the validation loss stops improving we stop training early, and we keep the version that did best on validation. This stops the network from simply memorising the training data.
- **Testing (15%)** is used only once, after training is finished, to give the final score.

The split is stratified, which means every modulation (and, for the Pluto data, every power level) appears in all three parts in the same proportion.

For the first stage we used the RadioML 2016.10a dataset from DeepSig. It contains eleven modulation classes of simulated I/Q frames. We kept our five classes at an SNR of 10 dB or more, which left 25,000 frames. We trained with the Adam optimiser, a learning rate of 0.001, mini-batches of 128 and a maximum of 30 epochs.

Set	RadioML frames	RadioML accuracy	Pluto frames	Pluto accuracy
Training	17,500	84.9%	10,500 + 5,000 RadioML	99.7%
Validation	3,750	83.3%	2,250	99.5%
Testing	3,750	**85.25%**	2,250	**99.6%**

In both stages the training, validation and test accuracies are within one or two points of each other. This tells us the network learned general patterns rather than memorising its training data. For the record, our mid-review model scored 85.86%, but it used an 80/20 split where the test set also served as the validation set. Retraining with a proper separate validation set gave 85.25%, and that is the figure we use throughout this report.

10. Real-World Dataset and Fine-Tuning

To teach the network what real signals look like, we recorded our own dataset with the Pluto. We transmitted each of the five modulations at five power levels and saved 1,000 frames at each level, 25,000 frames in total. To estimate the SNR of each level, we first recorded the noise with nothing transmitted and compared the signal power against it.

TX offset	TX gain	Estimated SNR	Used for
0 dB	−30 dB	37.8 dB	Training, validation and testing
−10 dB	−40 dB	28.1 dB	Training, validation and testing
−20 dB	−50 dB	16.9 dB	Training, validation and testing
−30 dB	−60 dB	7.5 dB	Testing only
−40 dB	−70 dB	−2.6 dB	Testing only

For training, validation and testing we used only the three strongest levels, which gives 15,000 frames (10,500, 2,250 and 2,250). The two weakest levels, another 10,000 frames, are close to the noise floor, so we kept them only for checking how accuracy changes with SNR. This mirrors the SNR ≥ 10 dB selection we made on RadioML.

Fine-tuning means we did not start again from scratch. We took the network already trained on RadioML, kept all its weights, and continued training it gently on real data:

- The training data was 10,500 real frames plus 5,000 RadioML training frames. We mixed in the RadioML frames so the network would not forget what it had learned from the dataset, and it still scores 83.2% on the RadioML test set afterwards.
- We used a learning rate of 0.0005, half the original, so the weights change in small steps. We tried 0.0001 as well and kept 0.0005 because it scored better on validation.
- Training ran for up to 30 epochs, with validation every 50 steps, early stopping and the best validation version kept.

Fine-tuning works because the network finally sees real examples of the effects that the simulated data lacks: the Pluto's filtering, IQ imbalance, phase rotation, reflections, real noise and the missing AM carrier.

11. Results

The main result is that the fine-tuned network is correct on 96.7% of an independent fresh recording, compared with 74.1% before fine-tuning. The fresh recording contains 9,600 frames of new random data at the three strongest power levels (1,920 per modulation), and it was never used for training or validation.

Evaluation	Data	Accuracy
RadioML CNN on the RadioML test set	3,750 simulated frames	85.25%
RadioML CNN on the fresh Pluto recording	9,600 real frames	74.1%
Fine-tuned CNN on the Pluto test split	2,250 real frames	99.6%
Fine-tuned CNN on the fresh Pluto recording	9,600 real frames	**96.7%**
Fine-tuned CNN on the RadioML test set	3,750 simulated frames	83.2%

Accuracy also depends strongly on signal strength, just as it does on RadioML. Above about 17 dB SNR the fine-tuned network is right 99 to 100% of the time. At the weakest level the signal is below the noise and both networks are close to chance.

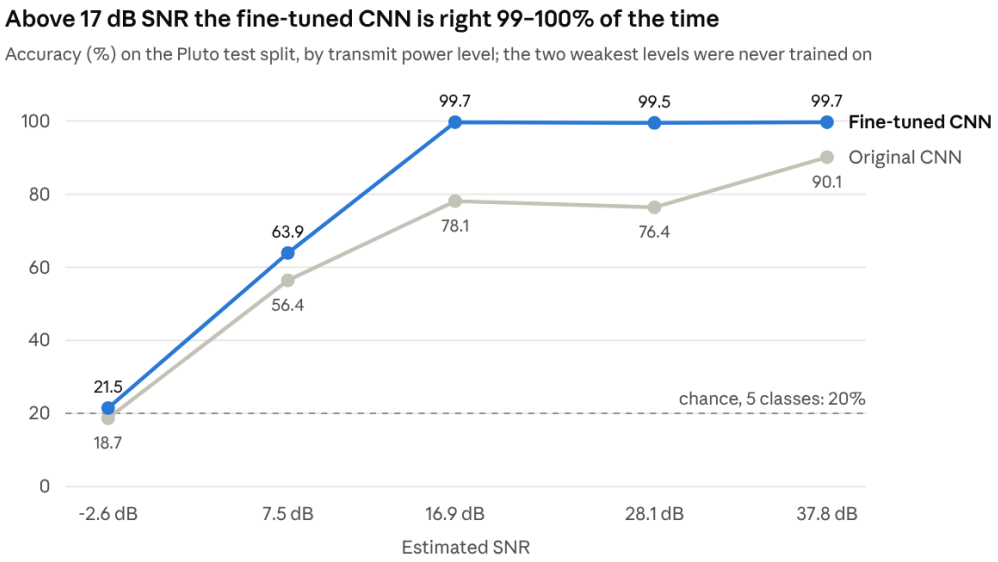


Figure 6. Accuracy against SNR on the Pluto test split, for the original and fine-tuned networks.

Confusion matrices

The confusion matrices show where the mistakes happen. Each row is the modulation that was actually sent and each column is what the network guessed, so the diagonal holds the correct answers. Before fine-tuning, AM-DSB was almost always wrong: only 84 of 1,920 frames were correct, and most were taken for QAM16 or WBFM. WBFM was also often mistaken for AM-DSB.

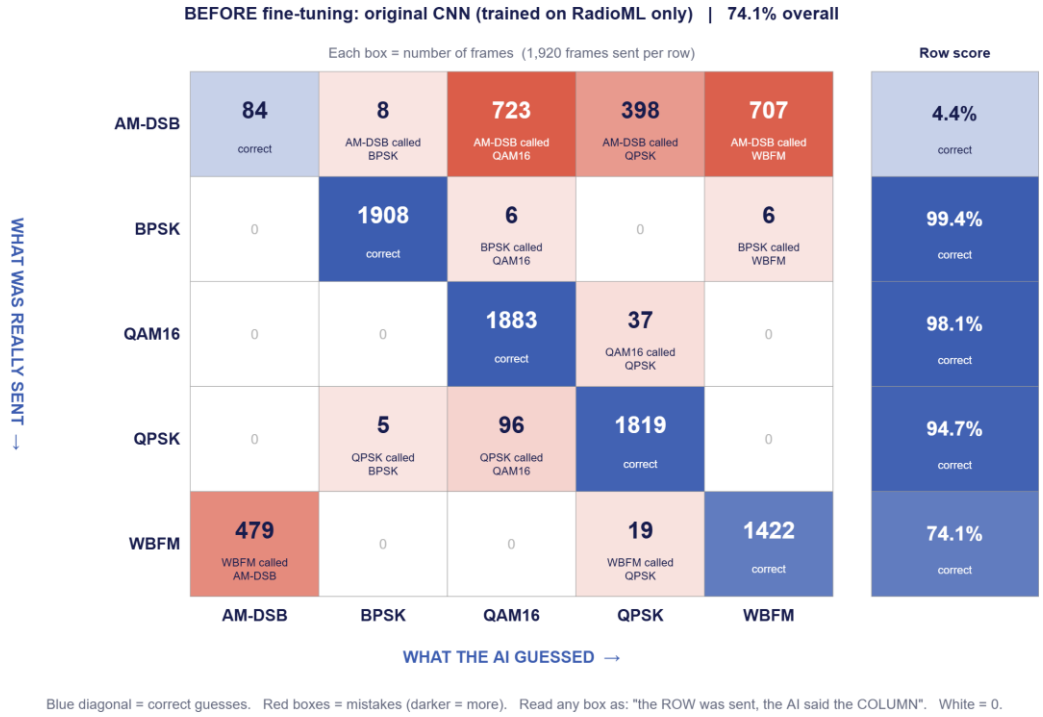


Figure 7. Confusion matrix before fine-tuning (74.1%).

After fine-tuning, the diagonal is strong for every class and 9,287 of the 9,600 frames are correct. AM-DSB went from 84 to 1,747 correct frames, WBFM from 1,422 to 1,830, and QPSK from 1,819 to 1,917.

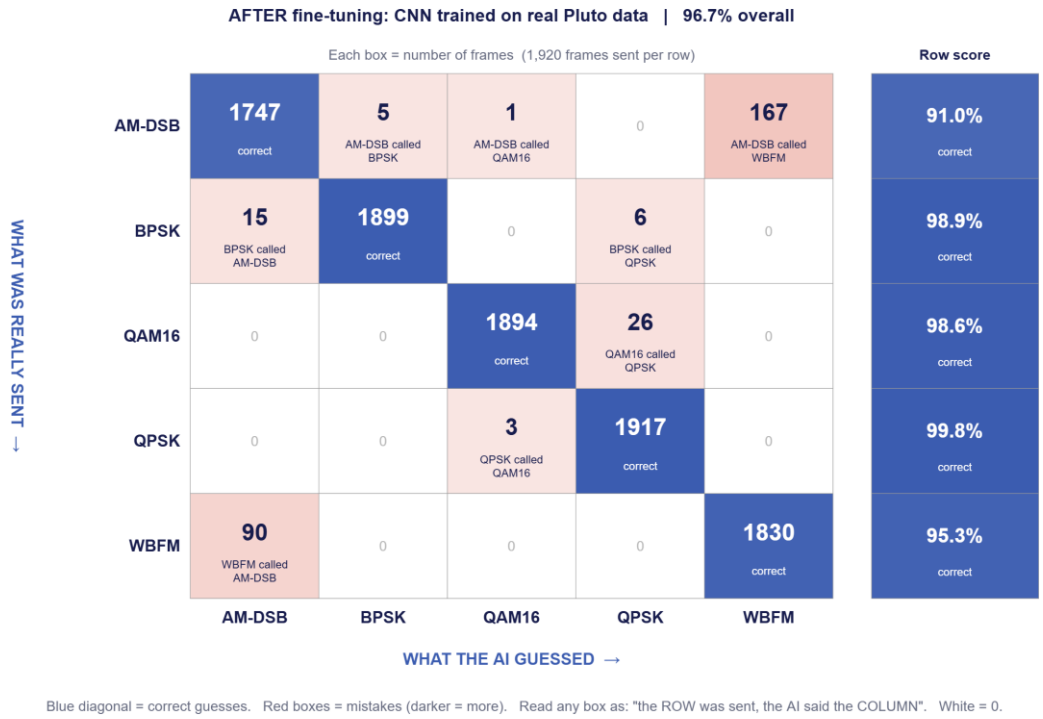


Figure 8. Confusion matrix after fine-tuning (96.7%).

Modulation sent	Correct before	Correct after	Main remaining mistake
AM-DSB	84 (4.4%)	1,747 (91.0%)	167 taken for WBFM
BPSK	1,908 (99.4%)	1,899 (98.9%)	15 taken for AM-DSB
QAM16	1,883 (98.1%)	1,894 (98.6%)	26 taken for QPSK
QPSK	1,819 (94.7%)	1,917 (99.8%)	3 taken for QAM16
WBFM	1,422 (74.1%)	1,830 (95.3%)	90 taken for AM-DSB

Deciding over more frames

A single frame is only 128 samples, or 0.13 ms of signal, which is very short. When we let the system take a majority vote over several frames in a row, the remaining mistakes disappear. With one frame the accuracy is 96.7%, with eight frames (1 ms) it is 99.5%, and with 64 frames (8 ms) it reaches 100%. Eight milliseconds is still instant for a person, and it is exactly how our live dashboard decides.

Live demonstration

The live dashboard shows the received I/Q frame, the spectrum, the constellation, the CNN's confidence for each class and the running frame accuracy. Pressing the keys 1 to 5 switches the transmitted modulation. In our live test with the antennas, all five modulations were recognised correctly. In the run shown below, QPSK was predicted with 96% confidence and 99.5% of frames were correct.

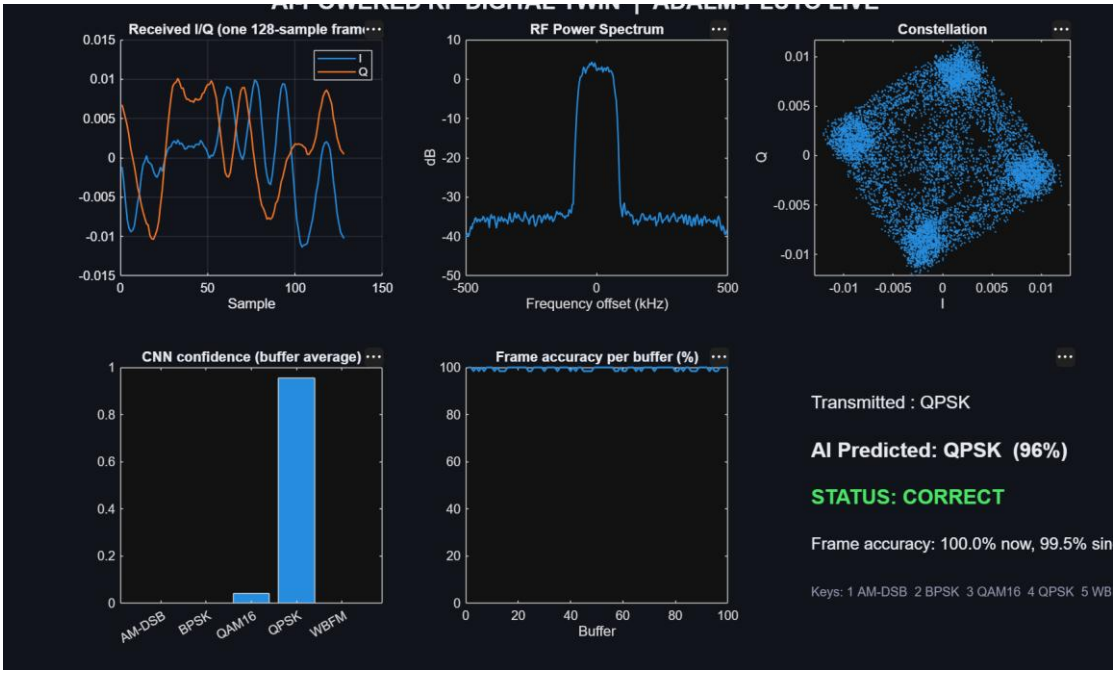


Figure 9. The live RF digital twin dashboard while QPSK is being transmitted.

12. Discussion

Fine-tuning clearly closed the gap on our own link. Still, we want to be careful about how the numbers are compared. The 85.25% on RadioML and the 96.7% on our hardware come from very different conditions. RadioML covers many simulated channels with fading, offsets and noise, while our test is one short-range lab link with a shared clock, and the network was fine-tuned on that same link. So we do not claim that real hardware beats simulation. What we do claim is that fine-tuning on real data raised the real-signal accuracy from 74.1% to 96.7%.

The analog classes gained the most: AM-DSB by 87 points and WBFM by 21 points. These are the signals that differ most between simulation and our hardware, partly because our DC removal strips away the AM carrier. The digital classes were already above 94% before fine-tuning.

The mistakes that remain make physical sense. AM-DSB and WBFM both carry the same music, so during a quiet moment a 0.13 ms frame of either one looks like an almost constant carrier. QPSK and QAM16 overlap because the four QPSK points sit where the corner points of the QAM16 grid are.

We also report the test split (99.6%) beside the fresh recording (96.7%) for a reason. The transmitted waveforms repeat every few milliseconds, so frames in the test split can look similar to frames used in training. The fresh recording uses new random data, which makes it the fairer number, and that is why we use it as our headline result.

Our work has some limits. We used one board for both transmitting and receiving, so there was no carrier frequency offset to deal with. We tested at one location and one short distance, with five classes, and with a single music clip as the only analog message.

13. Challenges We Faced

Most of our difficulties were in getting the radio link and the data pipeline right rather than in the network itself.

Challenge	Cause	How we solved it
The laptop could not see the Pluto	The cable was in the power-only micro-USB port	Moved it to the middle (data) port and installed the Analog Devices USB driver
MATLAB toolbox installs kept failing	Downloads dropped on the campus network	Installed with the MathWorks package manager over a stable connection
The receiver clipped once the antennas were fitted	Antennas only 2 cm apart give a very strong signal	Ran a gain sweep and settled on TX −30 dB and RX 30 dB
The signal was only about one ADC step tall, and frames of the previous modulation leaked into the next	Starting the transmitter reconfigured the radio chip and reset the receiver gain, and the receiver kept old buffers	Restarted the receiver after every transmitter start; the signal rose from about 1 to about 91 ADC steps
Low accuracy on real signals (74.1%)	Hardware effects that RadioML does not contain	Fine-tuned on 10,500 real frames, reaching 96.7%
Making the evaluation fair	Repeating waveforms make test frames resemble training frames	Added a separate validation set and tested on a fresh recording with new random data
Old files kept coming back	The project folder was inside OneDrive, which restored older versions	Paused syncing, recorded the data again and kept a backup outside OneDrive

14. Conclusion and Future Scope

We set out to show that a simple CNN for modulation classification can be taken off the dataset and made to work on real radio hardware, and we believe the results support that. The same network that scored 85.25% on RadioML dropped to 74.1% on real signals from the ADALM-Pluto, and after fine-tuning on our own recordings it reached 96.7% on a completely new recording, while running live in real time.

In summary, we:

- built an RF digital twin in which the same CNN runs on simulated and on real over-the-air signals,
- recorded 25,000 real frames for five modulations at five power levels,
- used stratified 70/15/15 training, validation and test splits in both stages,
- measured the gap between simulation and reality and closed it on our link with fine-tuning, and
- demonstrated live classification of all five modulations.

There are several ways this work could be taken further:

1. Add more modulations, such as 8PSK, GFSK, OFDM and the AM variants (DSB-SC and SSB) from the RadioML 2018.01A dataset. For the AM variants the preprocessing would need to keep the carrier.
2. Use two separate Plutos for transmitting and receiving, which would require frequency offset correction.
3. Classify real signals already present in the ISM bands, not only our own transmissions.
4. Run the CNN directly on the Pluto's FPGA using HDL Coder.
5. Build a real-time Simulink model using the Pluto Simulink blocks.
6. Try deeper networks such as ResNet or LSTM to improve accuracy below 5 dB SNR.

References

1. "Automatic Modulation Classification with Deep Neural Networks," *Electronics* (MDPI), 2023.
2. "Real-time Modulation Classification Architecture in Software Defined Radio," *Physical Communication* (ScienceDirect), 2024.
3. "Modulation Recognition Based on Multi-modal Fusion of ResNet and Traditional Signal Processing," IEEE MTT-S IWS (IEEE Xplore), 2025.
4. "A Novel Deep Learning Automatic Modulation Classifier with Fusion of Multichannel Information Using GRU," Springer Nature, 2023.
5. DeepSig Inc., RadioML 2016.10a dataset.
6. Analog Devices, ADALM-PLUTO Software-Defined Radio Active Learning Module documentation.
7. MathWorks, Communications Toolbox Support Package for ADALM-PLUTO Radio documentation.